

Practical Trade-offs of Priority Queue Implementations in Dijkstra’s and Prim’s Algorithms

Chenchen Wang

Yongjia Wang

Abstract

This project studies the practical trade-offs among binary heaps, pairing heaps, and Fibonacci heaps when used as priority queue backends for Dijkstra’s shortest-path algorithm and Prim’s minimum-spanning-tree algorithm. Although Fibonacci heaps have strong amortized bounds for decrease-key operations, practical performance also depends on constant factors, memory layout, pointer chasing, and graph structure. We implement all three heaps under a shared priority queue interface and evaluate them on synthetic graphs, SNAP real-world graphs, and decrease-key stress graphs. A Python prototype records structural heap metrics, with comparison count used as the main structural metric in this report. A C++ benchmark measures median runtime and estimated priority queue memory usage. Our results show that binary heaps remain strong on ordinary workloads, pairing heaps become competitive on decrease-key-heavy inputs, and Fibonacci heaps do not automatically outperform simpler implementations despite their theoretical advantages.

1 Introduction

Priority queues are a core component of both Dijkstra’s shortest-path algorithm and Prim’s minimum-spanning-tree algorithm. In both algorithms, vertices are repeatedly extracted according to the smallest current key, and keys are updated when a better distance or lighter connecting edge is found. Therefore, the choice of priority queue can strongly affect the practical performance of the full graph algorithm.

The theoretical comparison between heaps does not always predict practical performance. Fibonacci heaps support $O(1)$ amortized decrease-key operations and give the classical $O(E + V \log V)$ bound for graph algorithms such as Dijkstra’s algorithm. Binary heaps have weaker asymptotic bounds for decrease-key operations, but they are simple, array-based, and memory-local. Pairing heaps lie between these two designs: they use pointer-based cut-and-meld operations and are often considered a practical alternative to Fibonacci heaps. This project investigates when these different design choices matter in practice.

We compare three priority queue implementations: binary heap, pairing heap, and Fibonacci heap. Each heap is used as an interchangeable backend for the same implementations of Dijkstra’s and Prim’s algorithms. This setup keeps the graph algorithm fixed while changing only the priority queue implementation, allowing us to isolate the effect of heap choice.

The study focuses on three questions. First, which heap performs best on ordinary synthetic and real-world graph workloads? Second, do decrease-key stress workloads change the relative performance of the heaps? Third, how do comparison count, runtime, and estimated priority queue memory explain the practical trade-offs among the implementations?

To answer these questions, we use both a Python prototype and a C++ benchmark. The Python prototype is used for correctness checking and structural instrumentation, with mean comparison

count serving as the main structural metric in this report. The C++ benchmark provides the main runtime and memory measurements, including median runtime and estimated priority queue memory. By combining these measurements, we evaluate not only which heap is faster, but also why the performance differs across workloads.

2 Background

Dijkstra’s algorithm maintains tentative shortest-path distances from a source vertex. Each iteration extracts the unfinalized vertex with minimum tentative distance and relaxes its outgoing edges. If a relaxation improves a neighbor’s distance, the neighbor’s priority queue key is decreased. Prim’s algorithm has the same priority-queue pattern, but the key represents the lightest known edge connecting a vertex to the growing tree. Thus, both algorithms use the same core operations: insert, extract-min, and decrease-key.

The decrease-key operation is the main reason to compare different heap implementations. On graphs with few successful key updates, a simple binary heap may perform best because of low overhead and good locality. On graphs with many successful key updates, heaps designed for cheaper decrease-key operations may become more competitive. Table 1 summarizes the expected trade-offs.

Table 1: Qualitative comparison of the priority queue implementations.

Heap type	Expected strength	Expected weakness
Binary heap	Simple array-based structure, low constant factors, good locality	$O(\log n)$ worst-case decrease-key and possible repeated swaps
Pairing heap	Practical cut-and-meld behavior; simpler than Fibonacci heap	Pointer overhead, weaker locality, and subtle amortized decrease-key analysis
Fibonacci heap	$O(1)$ amortized decrease-key and strong theoretical bounds	High constant factors, complex pointer structure, cuts, and consolidation

In our analysis, binary heap bounds are treated as worst-case per-operation bounds. Pairing heap is treated as a practical pointer-based alternative with logarithmic amortized bounds in a conservative theoretical comparison. Fibonacci heap provides the strongest standard amortized bounds, but its more complex structure may increase memory usage and constant-factor overhead. These differences motivate an empirical study rather than relying only on asymptotic complexity.

3 Implementation and Experimental Setup

We implemented binary heap, pairing heap, and Fibonacci heap as interchangeable priority queue backends for the same Dijkstra and Prim implementations. All three heaps support insert, extract-min, decrease-key, contains, and empty. The algorithms use true decrease-key operations rather than lazy reinsertion: when a better key is found, the vertex is updated in place if it is already present in the heap.

The binary heap is implemented with an array and a handle structure mapping vertices to heap positions. The pairing heap and Fibonacci heap are pointer-based implementations. Pairing heap uses cut-and-meld style decrease-key, while Fibonacci heap uses root lists, cuts, cascading cuts,

and consolidation. These implementation differences motivate comparing both structural cost and practical runtime.

The experiments use three types of graph inputs. The synthetic suite includes sparse, medium-density, dense, and superdense Erdős–Rényi graphs, as well as two-dimensional grid graphs. The SNAP suite includes `ca-GrQc`, `roadNet-PA`, and `roadNet-CA`. For the road-network datasets, the default configuration reads at most 50,000 raw SNAP edges and then uses the largest connected component, so these results should be interpreted as capped road-network workloads. The C++ benchmark also includes decrease-key stress graphs designed to repeatedly improve target vertices’ keys.

The main reported metrics are mean comparison count, median runtime, and estimated priority queue memory. Mean comparison count is collected by the Python prototype and measures heap-dependent internal comparison work. Median runtime is collected by the C++ benchmark and serves as the main performance metric. Estimated priority queue memory measures the memory footprint of the priority queue itself. We do not use process-level RSS as the main memory metric because it also includes graph storage, algorithm arrays, and baseline process memory. Correctness is checked by comparing Dijkstra distance checksums and Prim MST weights across heap backends.

4 Results

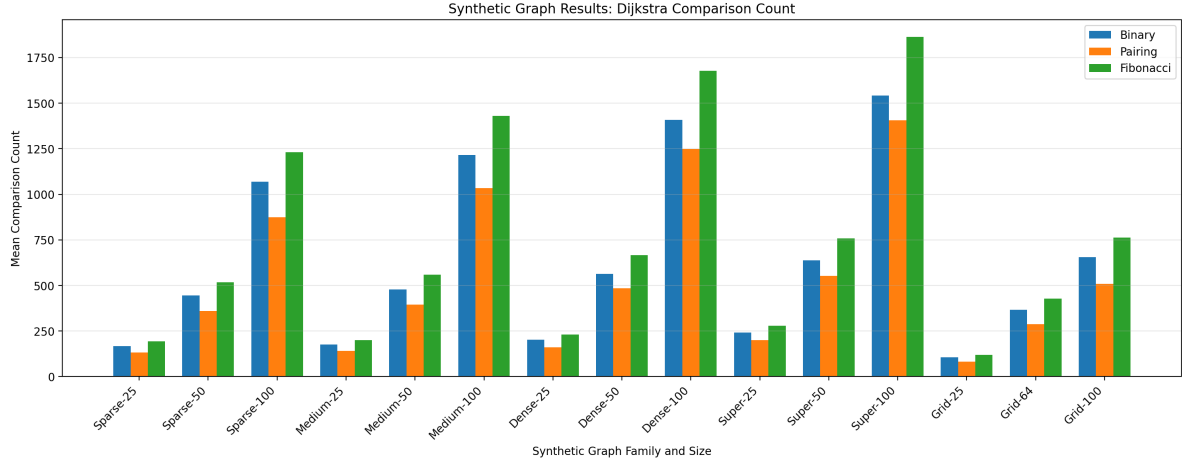
4.1 Synthetic Graph Results

We first evaluate the priority queues on synthetic graphs. These inputs provide a controlled setting for comparing heap behavior under different graph densities and structures. The synthetic graph families include sparse Erdős–Rényi graphs, medium-density Erdős–Rényi graphs, dense Erdős–Rényi graphs, superdense Erdős–Rényi graphs, and two-dimensional grid graphs.

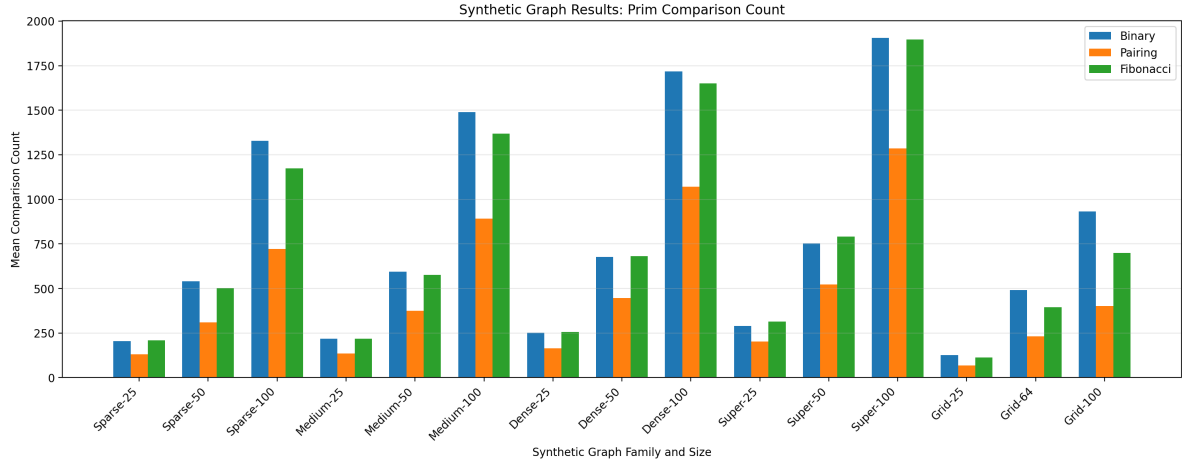
In this section, we focus on mean comparison count as the main Python structural metric. We do not analyze workload-level quantities in the main text because they are mostly determined by the graph and algorithm rather than by the heap implementation. Comparison count, in contrast, is heap-dependent: it measures how much internal key-comparison work each priority queue performs while processing the same algorithmic workload.

Figure 1 shows that the three heap implementations perform different amounts of internal comparison work. Pairing heap generally has the lowest mean comparison count across the synthetic graph families. For Dijkstra’s algorithm, Fibonacci heap often performs the most comparisons, while binary heap is usually between pairing heap and Fibonacci heap. For Prim’s algorithm, pairing heap again has the lowest comparison count, while binary heap and Fibonacci heap perform more comparison work.

These results suggest that the same graph workload can lead to substantially different internal heap costs. However, comparison count alone does not fully determine runtime. A heap may perform fewer comparisons but still have higher runtime due to pointer-heavy structure, additional bookkeeping, or weaker memory locality. For that reason, the Python comparison-count results are best interpreted as structural evidence, while the C++ benchmark in Section 4.3 provides the main runtime evidence.



(a) Dijkstra's algorithm.



(b) Prim's algorithm.

Figure 1: Mean comparison count on synthetic graphs. Each group contains three bars corresponding to binary heap, pairing heap, and Fibonacci heap. This metric is heap-dependent and shows how different priority queue structures translate the same graph workload into different internal comparison costs.

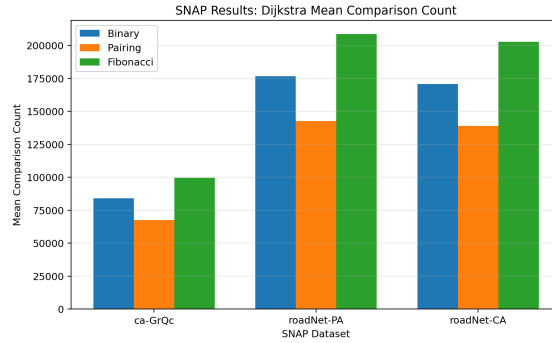
4.2 SNAP Results

We next evaluate the priority queues on three real-world SNAP graph inputs: **ca-GrQc**, **roadNet-PA**, and **roadNet-CA**. These datasets represent different graph structures. **ca-GrQc** is a collaboration network, while **roadNet-PA** and **roadNet-CA** are road-network inputs. The road-network experiments use capped subgraphs by default, so these results should be interpreted as results on sampled road-network workloads rather than on the complete original road networks.

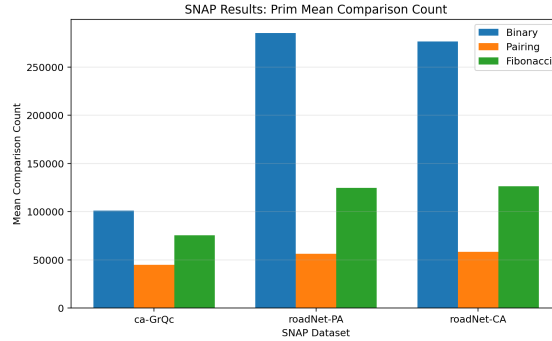
As with the synthetic experiments, we focus on mean comparison count for the SNAP summaries. This keeps the SNAP analysis centered on heap-dependent behavior rather than quantities that are mostly shared across heap implementations.

Figure 2 shows a similar pattern on real-world graphs. For Dijkstra’s algorithm, pairing heap has the lowest comparison count on all three SNAP datasets, while Fibonacci heap has the highest comparison count. Binary heap falls between them. For Prim’s algorithm, the difference is especially visible on the road-network inputs: binary heap performs substantially more comparisons than pairing heap and Fibonacci heap on both **roadNet-PA** and **roadNet-CA**.

The SNAP comparison-count results reinforce the main observation from the synthetic experiments: different priority queue implementations can incur very different internal structural costs even when they solve the same graph problem. At the same time, comparison count should not be treated as the final performance metric. The C++ runtime results below are needed to determine whether lower comparison counts translate into faster execution.



(a) Dijkstra’s algorithm.



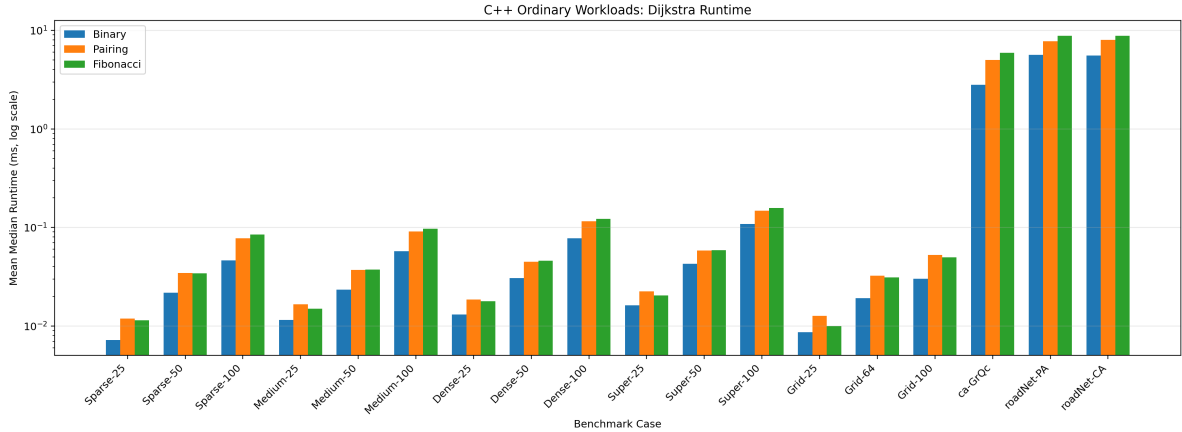
(b) Prim’s algorithm.

Figure 2: Mean comparison count on SNAP graphs. Each dataset group contains three bars corresponding to binary heap, pairing heap, and Fibonacci heap. Comparison count is heap-dependent and measures the internal key-comparison work performed by each priority queue implementation.

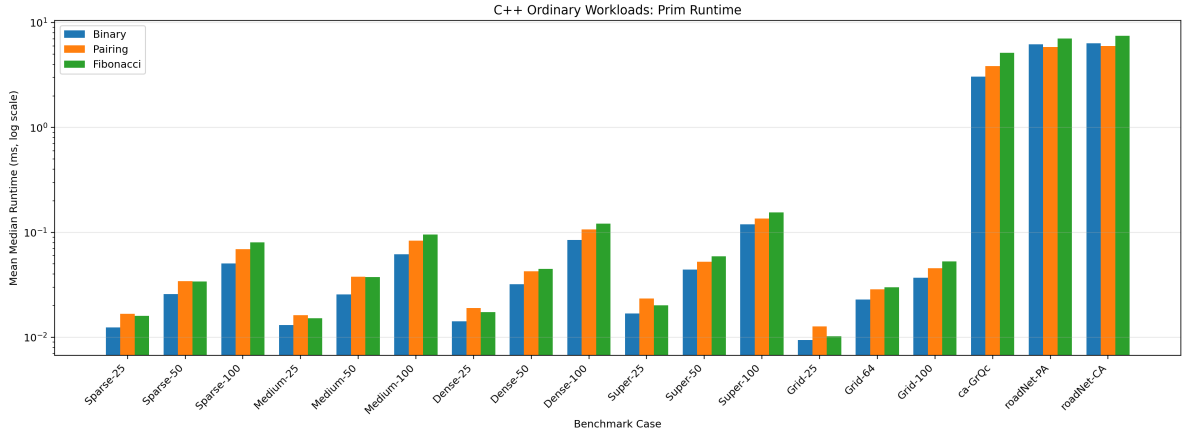
4.3 C++ Runtime and Memory Results

The Python summaries above help explain structural differences among the heap implementations. We now use the C++ benchmark as the main evidence for practical runtime and memory behavior. In this section, runtime refers to `mean_median_runtime_ms`, the mean of per-run median runtimes, and memory refers to `mean_priority_queue_memory_bytes`, converted to KiB in the figures. These metrics capture practical running time and heap-level memory overhead.

Figure 3 reports the C++ median runtime on ordinary synthetic and SNAP workloads. The decrease-key stress graphs are excluded from this figure and analyzed separately. The y-axis uses a logarithmic scale because the small synthetic graphs have much shorter runtimes than the SNAP inputs. Within each benchmark case, the three bars correspond to binary heap, pairing heap, and Fibonacci heap.



(a) Dijkstra's algorithm.



(b) Prim's algorithm.

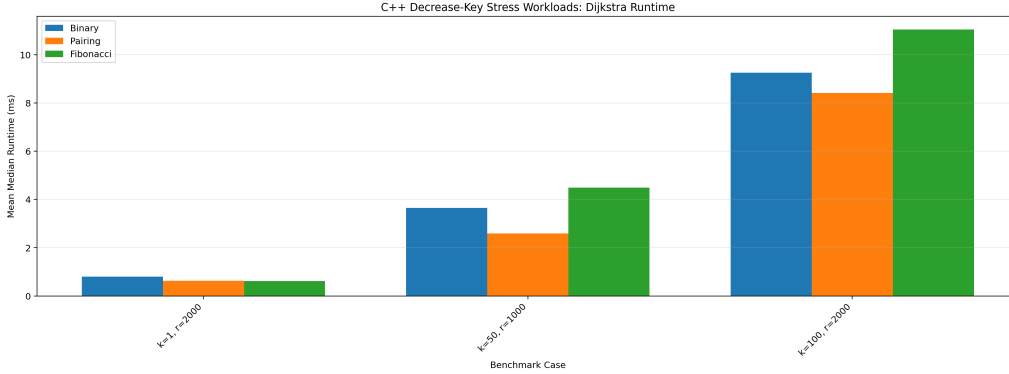
Figure 3: C++ median runtime on ordinary synthetic and SNAP workloads. Stress graphs are excluded. The y-axis uses a logarithmic scale because the runtime range differs substantially between small synthetic graphs and SNAP graphs.

On ordinary workloads, binary heap is usually the fastest or close to the fastest implementation. This is especially clear for Dijkstra's algorithm, where binary heap has the lowest runtime on the synthetic graph families and on all three SNAP inputs. This suggests that the low constant factors

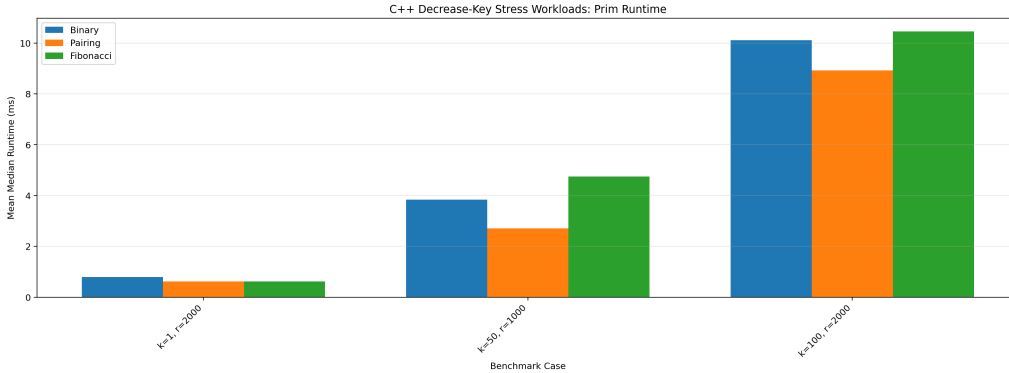
and contiguous array layout of binary heap are very effective in ordinary graph workloads.

Prim shows a slightly different pattern. Binary heap remains strong on the synthetic inputs, but pairing heap becomes competitive on the road-network SNAP inputs. On **roadNet-PA** and **roadNet-CA**, pairing heap has lower median runtime than binary heap. This result suggests that pairing heap can be a practical alternative in some workloads, even though binary heap is often the best default choice.

Figure 4 reports C++ runtime on the decrease-key stress graphs. These graphs are designed to produce many repeated key improvements, which makes them useful for testing whether pairing heap and Fibonacci heap benefit from update-heavy workloads.



(a) Dijkstra stress graphs.



(b) Prim stress graphs.

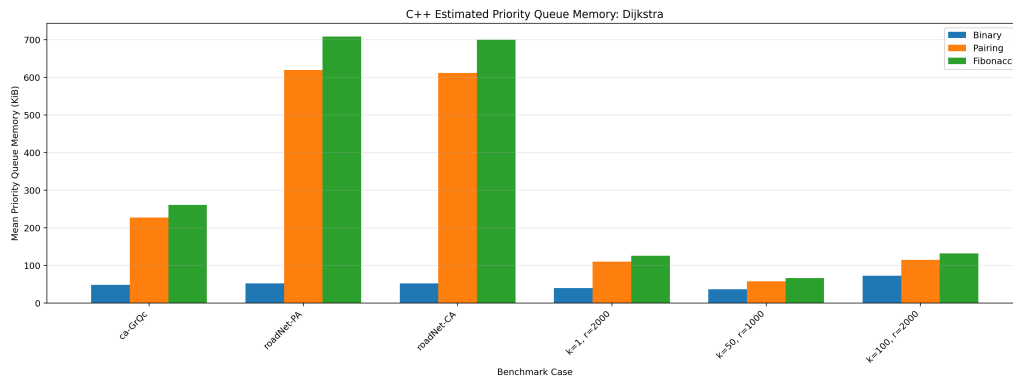
Figure 4: C++ median runtime on decrease-key stress graphs. Each stress case is parameterized by the number of improver vertices k and target vertices r . These inputs are designed to generate many successful decrease-key operations.

The stress-graph results show that heap choice changes when the workload is constructed to emphasize repeated key updates. Pairing heap performs especially well on the larger stress cases. For Dijkstra, pairing heap is faster than binary heap and Fibonacci heap on the $k = 50, r = 1000$ and $k = 100, r = 2000$ stress graphs. The same pattern appears for Prim. This supports the idea that pairing heap can serve as a practical compromise: it avoids the repeated array movement of binary heap during many key updates, while avoiding some of the larger constant-factor overheads of Fibonacci heap.

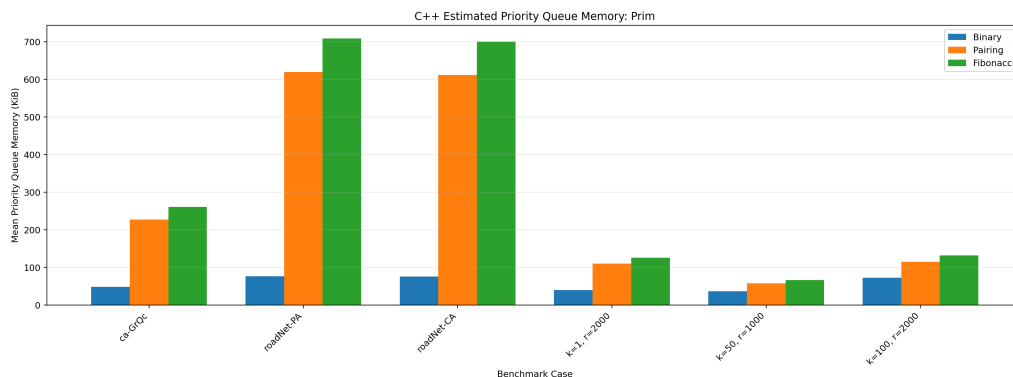
Fibonacci heap is fastest on the small $k = 1, r = 2000$ stress case, but it does not remain the fastest across all stress inputs. On the larger stress cases, Fibonacci heap is often slower than

pairing heap and sometimes slower than binary heap. This illustrates the gap between amortized theoretical guarantees and practical performance. Although Fibonacci heap has strong amortized bounds, its pointer-heavy structure, cuts, cascading cuts, and consolidation overhead can offset its theoretical advantage.

Figure 5 reports estimated priority queue memory usage. We plot memory for the SNAP inputs and the stress graphs, since these larger cases are more informative than the smallest synthetic graphs. The values are reported in KiB and correspond to estimated priority queue memory rather than whole-process RSS.



(a) Dijkstra's algorithm.



(b) Prim's algorithm.

Figure 5: Estimated priority queue memory usage in the C++ benchmark. The figure shows heap-level memory estimates for SNAP inputs and decrease-key stress graphs. Values are reported in KiB.

The memory results show a clear difference between array-based and pointer-based priority queues. Binary heap uses the least priority queue memory on most larger cases because it stores heap entries compactly in arrays. Pairing heap and Fibonacci heap require node-based structures and handle arrays, which increase memory usage. Fibonacci heap often has the largest memory footprint because each node stores additional structural information such as degree, parent and child pointers, sibling links, and mark information.

We do not use process-level RSS as the main memory figure because RSS measures the memory footprint of the entire benchmark process, including graph storage, algorithm arrays, process baseline memory, and the priority queue. Therefore, RSS is useful as a whole-program memory measurement, but it is not a heap-only metric. For comparing the memory overhead of binary

heap, pairing heap, and Fibonacci heap, estimated priority queue memory is the more relevant measurement.

5 Discussion and Conclusion

The results show that asymptotic complexity alone does not determine practical priority queue performance in Dijkstra’s and Prim’s algorithms. Fibonacci heap has the strongest amortized decrease-key guarantee among the three implementations, but it is not consistently the fastest in our experiments. On ordinary synthetic and SNAP workloads, especially for Dijkstra’s algorithm, binary heap is usually the fastest or close to the fastest implementation. This suggests that low constant factors, compact array-based storage, and good memory locality can outweigh the theoretical advantages of more complex pointer-based heaps.

The comparison-count results help explain part of this gap between theory and practice. Pairing heap often performs fewer comparisons than binary heap or Fibonacci heap, especially for Prim and for the SNAP road-network inputs. However, fewer comparisons do not automatically imply lower runtime. Pairing heap and Fibonacci heap also pay overhead from node structures, pointer updates, cuts, links, and weaker locality. Therefore, comparison count is useful as a structural metric, but the C++ runtime results are the main evidence for practical performance.

The C++ stress-graph results show that heap rankings can change when the workload is designed to create many key improvements. On the larger decrease-key stress cases, pairing heap outperforms both binary heap and Fibonacci heap for both Dijkstra and Prim. This supports the view that pairing heap is a practical middle ground: it avoids the repeated upward array movement of binary heap during many decrease-key operations, while also avoiding some of the larger constant-factor overheads of Fibonacci heap. Fibonacci heap is fastest on one smaller stress case, but it does not remain fastest across all stress inputs, illustrating that strong amortized bounds do not always overcome implementation overhead.

Memory usage further reinforces the distinction between array-based and pointer-based priority queues. Binary heap generally uses the least estimated priority queue memory because it stores heap entries compactly in arrays. Pairing heap and Fibonacci heap require node-based representations and handle structures, and Fibonacci heap often has the largest memory footprint because each node stores additional structural information such as degree, mark information, parent and child pointers, and sibling links. This memory overhead is part of the practical cost of theoretically more advanced heaps.

There are several limitations to this evaluation. The Python prototype is used mainly for structural measurements rather than final runtime conclusions. The SNAP road-network experiments use capped subgraphs, so the results should not be interpreted as full-road-network benchmarks. The C++ benchmark measures runtime and estimated priority queue memory, but it does not directly measure hardware-level effects such as cache misses, branch mispredictions, or allocation latency. Finally, the results depend on implementation choices such as node layout, handle representation, and memory management strategy.

Overall, binary heap is a strong practical baseline for ordinary graph workloads because of its simplicity and locality. Pairing heap becomes attractive when decrease-key operations are frequent, especially in stress workloads and some Prim cases. Fibonacci heap remains theoretically important, but its practical advantage is not automatic. The best heap choice depends on the interaction between graph structure, algorithm behavior, update frequency, runtime overhead, and memory usage.

Appendix

A Synthetic Graph Summary

Table 2: Complete grouped summary of the synthetic graph experiments from `syn_summary.csv`.

Algorithm	Graph Family	Size	Parameters	Heap	Rows	Mean Compare	Mean Cuts	Dec-key per Extract	Edge scan per Extract	Mean Links	Mean Melds	Peak Heap	Dec-key Rate	Relax Rate	Mean Swaps
dijkstra	dense_er_np	100	n=100,p=0.18	binary	30	1407.4	0	1.497	19.56	0	0	83.4	1	0.254	590
dijkstra	dense_er_np	100	n=100,p=0.18	fibonacci	30	1677.4	55.033	1.497	19.56	461.633	0	83.4	1	0.254	0
dijkstra	dense_er_np	100	n=100,p=0.18	pairing	30	1248.633	149.7	1.497	19.56	1098.9	1098.9	83.4	1	0.254	0
dijkstra	dense_er_np	25	n=25,p=0.35	binary	30	203.4	0	0.921	9.752	0	0	18.9	1	0.387	84.533
dijkstra	dense_er_np	25	n=25,p=0.35	fibonacci	30	231.333	8.467	0.921	9.752	59.733	0	18.9	1	0.387	0
dijkstra	dense_er_np	25	n=25,p=0.35	pairing	30	160.767	22.8	0.921	9.752	137.733	137.733	18.9	1	0.387	0
dijkstra	dense_er_np	50	n=50,p=0.25	binary	30	563.567	0	1.277	13.908	0	0	39.8	1	0.325	231.9
dijkstra	dense_er_np	50	n=50,p=0.25	fibonacci	30	666.4	24.267	1.277	13.908	176.767	0	39.8	1	0.325	0
dijkstra	dense_er_np	50	n=50,p=0.25	pairing	30	484.233	63.767	1.277	13.908	420.367	420.367	39.8	1	0.325	0
dijkstra	grid_2d	100	rows=10,cols=10	binary	30	655.9	0	0.265	3.6	0	0	16.9	1	0.697	280.433
dijkstra	grid_2d	100	rows=10,cols=10	fibonacci	30	763.133	9.367	0.265	3.6	235.7	0	16.9	1	0.697	0
dijkstra	grid_2d	100	rows=10,cols=10	pairing	30	508.3	25.767	0.265	3.6	481.833	481.833	16.9	1	0.697	0
dijkstra	grid_2d	25	rows=5,cols=5	binary	30	106.067	0	0.227	3.2	0	0	8.3	1	0.742	43.967
dijkstra	grid_2d	25	rows=5,cols=5	fibonacci	30	119.267	2.1	0.227	3.2	33.767	0	8.3	1	0.742	0
dijkstra	grid_2d	25	rows=5,cols=5	pairing	30	81.2	5.3	0.227	3.2	75.533	75.533	8.3	1	0.742	0
dijkstra	grid_2d	64	rows=8,cols=8	binary	30	367.2	0	0.252	3.5	0	0	12.867	1	0.706	155.033
dijkstra	grid_2d	64	rows=8,cols=8	fibonacci	30	427.633	5.067	0.252	3.5	131.267	0	12.867	1	0.706	0
dijkstra	grid_2d	64	rows=8,cols=8	pairing	30	287.967	15.633	0.252	3.5	271.867	271.867	12.867	1	0.706	0
dijkstra	medium_er_np	100	n=100,p=0.08	binary	30	1216.467	0	0.932	9.686	0	0	69.4	1	0.397	526.6
dijkstra	medium_er_np	100	n=100,p=0.08	fibonacci	30	1431.2	34.167	0.932	9.686	427.767	0	69.4	1	0.397	0
dijkstra	medium_er_np	100	n=100,p=0.08	pairing	30	1034.267	93.167	0.932	9.686	941.067	941.067	69.4	1	0.397	0
dijkstra	medium_er_np	25	n=25,p=0.2	binary	30	175.267	0	0.656	6.552	0	0	16.467	1	0.496	72.267
dijkstra	medium_er_np	25	n=25,p=0.2	fibonacci	30	200.5	5.967	0.656	6.552	52.667	0	16.467	1	0.496	0
dijkstra	medium_er_np	25	n=25,p=0.2	pairing	30	140.933	16.167	0.656	6.552	124.533	124.533	16.467	1	0.496	0
dijkstra	medium_er_np	50	n=50,p=0.12	binary	30	477.067	0	0.787	7.924	0	0	33.4	1	0.447	202.333
dijkstra	medium_er_np	50	n=50,p=0.12	fibonacci	30	559.3	15.7	0.787	7.924	161.133	0	33.4	1	0.447	0
dijkstra	medium_er_np	50	n=50,p=0.12	pairing	30	394.667	39.3	0.787	7.924	355.3	355.3	33.4	1	0.447	0
dijkstra	sparse_er_nm	100	n=100,m=300	binary	30	1070	0	0.562	6	0	0	55.867	1	0.517	474.167
dijkstra	sparse_er_nm	100	n=100,m=300	fibonacci	30	1232.033	21.3	0.562	6	383.967	0	55.867	1	0.517	0
dijkstra	sparse_er_nm	100	n=100,m=300	pairing	30	873.333	56.067	0.562	6	817.167	817.167	55.867	1	0.517	0
dijkstra	sparse_er_nm	25	n=25,m=75	binary	30	167.1	0	0.551	6	0	0	15.7	1	0.504	69.7
dijkstra	sparse_er_nm	25	n=25,m=75	fibonacci	30	193.233	4.833	0.551	6	52.867	0	15.7	1	0.504	0
dijkstra	sparse_er_nm	25	n=25,m=75	pairing	30	132.6	13.467	0.551	6	118.833	118.833	15.7	1	0.504	0
dijkstra	sparse_er_nm	50	n=50,m=150	binary	30	444.2	0	0.628	6	0	0	29.9	1	0.536	189.933
dijkstra	sparse_er_nm	50	n=50,m=150	fibonacci	30	516.967	12.2	0.628	6	150.067	0	29.9	1	0.536	0

Table 2: Complete grouped summary of the synthetic graph experiments from `syn_summary.csv`. Continued.

Algorithm	Graph Family	Size	Parameters	Heap	Rows	Mean Compare	Mean Cuts	Dec-key per Extract	Edge scan per Extract	Mean Links	Mean Melds	Peak Heap	Dec-key Rate	Relax Rate	Mean Swaps
dijkstra	sparse_er_nm	50	n=50,m=150	pairing	30	359.533	31.2	0.628	6	328.133	328.133	29.9	1	0.536	0
dijkstra	superdense_er_np	100	n=100,p=0.35	binary	30	1542.567	0	1.947	36.186	0	0	90.1	1	0.162	630.567
dijkstra	superdense_er_np	100	n=100,p=0.35	fibonacci	30	1864.233	72.667	1.947	36.186	487.667	0	90.1	1	0.162	0
dijkstra	superdense_er_np	100	n=100,p=0.35	pairing	30	1405.767	194.667	1.947	36.186	1211.1	1211.1	90.1	1	0.162	0
dijkstra	superdense_er_np	25	n=25,p=0.65	binary	30	242.467	0	1.413	16.472	0	0	21.667	1	0.288	96.2
dijkstra	superdense_er_np	25	n=25,p=0.65	fibonacci	30	279.367	13.067	1.413	16.472	63.833	0	21.667	1	0.288	0
dijkstra	superdense_er_np	25	n=25,p=0.65	pairing	30	199.433	35.167	1.413	16.472	164.1	164.1	21.667	1	0.288	0
dijkstra	superdense_er_np	50	n=50,p=0.5	binary	30	637.6	0	1.727	25.628	0	0	44.433	1	0.211	257.067
dijkstra	superdense_er_np	50	n=50,p=0.5	fibonacci	30	757.467	34.2	1.727	25.628	187.067	0	44.433	1	0.211	0
dijkstra	superdense_er_np	50	n=50,p=0.5	pairing	30	553.467	86.333	1.727	25.628	467.1	467.1	44.433	1	0.211	0
prim	dense_er_np	100	n=100,p=0.18	binary	10	1717.3	0	2.07	19.56	0	0	83.4	1	–	831.2
prim	dense_er_np	100	n=100,p=0.18	fibonacci	10	1650.2	127.4	2.07	19.56	364	0	83.4	1	–	0
prim	dense_er_np	100	n=100,p=0.18	pairing	10	1070.5	206.2	2.07	19.56	863.5	863.5	83.4	1	–	0
prim	dense_er_np	25	n=25,p=0.35	binary	10	252.7	0	1.488	9.752	0	0	18.9	1	–	114.3
prim	dense_er_np	25	n=25,p=0.35	fibonacci	10	256.2	19.5	1.488	9.752	51.2	0	18.9	1	–	0
prim	dense_er_np	25	n=25,p=0.35	pairing	10	163.9	36.3	1.488	9.752	126.7	126.7	18.9	1	–	0
prim	dense_er_np	50	n=50,p=0.25	binary	10	676.2	0	1.788	13.908	0	0	40.1	1	–	312.1
prim	dense_er_np	50	n=50,p=0.25	fibonacci	10	680.8	52.2	1.788	13.908	146.2	0	40.1	1	–	0
prim	dense_er_np	50	n=50,p=0.25	pairing	10	446.3	88.1	1.788	13.908	356.9	356.9	40.1	1	–	0
prim	grid_2d	100	rows=10,cols=10	binary	10	931.7	0	0.562	3.6	0	0	25.7	1	–	505.6
prim	grid_2d	100	rows=10,cols=10	fibonacci	10	698.3	34.5	0.562	3.6	154.6	0	25.7	1	–	0
prim	grid_2d	100	rows=10,cols=10	pairing	10	400.9	52.6	0.562	3.6	344.7	344.7	25.7	1	–	0
prim	grid_2d	25	rows=5,cols=5	binary	10	126	0	0.44	3.2	0	0	8.9	1	–	64.3
prim	grid_2d	25	rows=5,cols=5	fibonacci	10	112.7	6.1	0.44	3.2	21.7	0	8.9	1	–	0
prim	grid_2d	25	rows=5,cols=5	pairing	10	69.2	9.3	0.44	3.2	58.2	58.2	8.9	1	–	0
prim	grid_2d	64	rows=8,cols=8	binary	10	491.5	0	0.509	3.5	0	0	17	1	–	260.6
prim	grid_2d	64	rows=8,cols=8	fibonacci	10	393.9	19.6	0.509	3.5	85.2	0	17	1	–	0
prim	grid_2d	64	rows=8,cols=8	pairing	10	232.3	30	0.509	3.5	199.7	199.7	17	1	–	0
prim	medium_er_np	100	n=100,p=0.08	binary	10	1489.1	0	1.429	9.686	0	0	69.2	1	–	741.8
prim	medium_er_np	100	n=100,p=0.08	fibonacci	10	1369.3	90.4	1.429	9.686	324.3	0	69.2	1	–	0
prim	medium_er_np	100	n=100,p=0.08	pairing	10	891.8	141.7	1.429	9.686	748.9	748.9	69.2	1	–	0
prim	medium_er_np	25	n=25,p=0.2	binary	10	218.8	0	1.116	6.552	0	0	16.4	1	–	103
prim	medium_er_np	25	n=25,p=0.2	fibonacci	10	218.5	16.5	1.116	6.552	45.6	0	16.4	1	–	0
prim	medium_er_np	25	n=25,p=0.2	pairing	10	134.9	26.5	1.116	6.552	107	107	16.4	1	–	0
prim	medium_er_np	50	n=50,p=0.12	binary	10	593.6	0	1.346	7.924	0	0	34.2	1	–	281.7
prim	medium_er_np	50	n=50,p=0.12	fibonacci	10	577.2	38.6	1.346	7.924	129.9	0	34.2	1	–	0
prim	medium_er_np	50	n=50,p=0.12	pairing	10	374.7	66	1.346	7.924	307.4	307.4	34.2	1	–	0
prim	sparse_er_nm	100	n=100,m=300	binary	10	1328.9	0	1.022	6	0	0	56.2	1	–	681.8
prim	sparse_er_nm	100	n=100,m=300	fibonacci	10	1172.7	72	1.022	6	286.7	0	56.2	1	–	0
prim	sparse_er_nm	100	n=100,m=300	pairing	10	720.7	101.2	1.022	6	618.5	618.5	56.2	1	–	0
prim	sparse_er_nm	25	n=25,m=75	binary	10	203.7	0	0.976	6	0	0	15.5	1	–	94.3
prim	sparse_er_nm	25	n=25,m=75	fibonacci	10	209.7	13	0.976	6	46.2	0	15.5	1	–	0
prim	sparse_er_nm	25	n=25,m=75	pairing	10	130.1	23.2	0.976	6	105.7	105.7	15.5	1	–	0
prim	sparse_er_nm	50	n=50,m=150	binary	10	539.5	0	1.012	6	0	0	30.8	1	–	269.1

Table 2: Complete grouped summary of the synthetic graph experiments from `syn_summary.csv`. Continued.

Algorithm	Graph Family	Size	Parameters	Heap	Rows	Mean Compare	Mean Cuts	Dec-key per Extract	Edge scan per Extract	Mean Links	Mean Melds	Peak Heap	Dec-key Rate	Relax Rate	Mean Swaps
prim	sparse_er_nm	50	n=50,m=150	fibonacci	10	501.4	32.9	1.012	6	117.2	0	30.8	1	–	0
prim	sparse_er_nm	50	n=50,m=150	pairing	10	310.8	49.3	1.012	6	260.2	260.2	30.8	1	–	0
prim	superdense_er_np	100	n=100,p=0.35	binary	10	1906.3	0	2.693	36.186	0	0	91	1	–	886.3
prim	superdense_er_np	100	n=100,p=0.35	fibonacci	10	1896.4	156.2	2.693	36.186	396.3	0	91	1	–	0
prim	superdense_er_np	100	n=100,p=0.35	pairing	10	1286.6	268.3	2.693	36.186	1017.3	1017.3	91	1	–	0
prim	superdense_er_np	25	n=25,p=0.65	binary	10	290.7	0	2	16.472	0	0	21.8	1	–	124.9
prim	superdense_er_np	25	n=25,p=0.65	fibonacci	10	315.2	25.9	2	16.472	61	0	21.8	1	–	0
prim	superdense_er_np	25	n=25,p=0.65	pairing	10	202.7	48.8	2	16.472	152.7	152.7	21.8	1	–	0
prim	superdense_er_np	50	n=50,p=0.5	binary	10	753.1	0	2.33	25.628	0	0	44.4	1	–	332.8
prim	superdense_er_np	50	n=50,p=0.5	fibonacci	10	790	62.1	2.33	25.628	160.7	0	44.4	1	–	0
prim	superdense_er_np	50	n=50,p=0.5	pairing	10	521.5	115.3	2.33	25.628	405	405	44.4	1	–	0

B ca-GrQc Summary

Graph family: snap_ca-GrQc; *size:* 4158; *parameters:* dataset=ca-GrQc,component=largest,max_edges=50000.

Table 3: Complete grouped summary of the **ca-GrQc** experiments from `ca-GrQc_summary.csv`. Repeated graph metadata is shown above to keep the numeric columns readable.

Algorithm	Heap	Rows	Mean Compare	Mean Cuts	Dec-key per Extract	Edge scan per Extract	Mean Links	Mean Melds	Peak Heap	Dec-key Rate	Relax Rate	Mean Swaps
dijkstra	binary	30	84040.533	0	0.536	6.456	0	0	1499.433	1	0.476	39015.9
dijkstra	fibonacci	30	99587.567	858.6	0.536	6.456	35241.667	0	1499.433	1	0.476	0
dijkstra	pairing	30	67671.8	2228.3	0.536	6.456	65443.267	65443.267	1499.433	1	0.476	0
prim	binary	10	101270.3	0	0.773	6.456	0	0	1457.1	1	–	55748.5
prim	fibonacci	10	75454.2	1929.7	0.773	6.456	22022	0	1457.1	1	–	0
prim	pairing	10	44965.2	3159.2	0.773	6.456	41749.6	41749.6	1457.1	1	–	0

C roadNet-PA Summary

Graph family: snap.roadNet-PA; *size:* 11325; *parameters:* dataset=roadNet-PA, component=largest, max_edges=50000.

Table 4: Complete grouped summary of the roadNet-PA experiments from roadNet-PA_summary.csv. Repeated graph metadata is shown above to keep the numeric columns readable.

Algorithm	Heap	Rows	Mean Compare	Mean Cuts	Dec-key per Extract	Edge scan per Extract	Mean Links	Mean Melds	Peak Heap	Dec-key Rate	Relax Rate	Mean Swaps
dijkstra	binary	30	176609.333	0	0.164	2.863	0	0	419.133	1	0.813	81392.933
dijkstra	fibonacci	30	208670.467	749.433	0.164	2.863	76034.2	0	419.133	1	0.813	0
dijkstra	pairing	30	142622.5	1857.4	0.164	2.863	140764.9	140764.9	419.133	1	0.813	0
prim	binary	10	285201.3	0	0.315	2.863	0	0	1299	1	–	175462.1
prim	fibonacci	10	124668.2	2580.4	0.315	2.863	27743.8	0	1299	1	–	0
prim	pairing	10	56376.5	3409	0.315	2.863	52807.6	52807.6	1299	1	–	0

D roadNet-CA Summary

Graph family: snap_roadNet-CA; *size:* 11183; *parameters:* dataset=roadNet-CA, component=largest, max_edges=50000.

Table 5: Complete grouped summary of the roadNet-CA experiments from roadNet-CA_summary.csv. Repeated graph metadata is shown above to keep the numeric columns readable.

Algorithm	Heap	Rows	Mean Compare	Mean Cuts	Dec-key per Extract	Edge scan per Extract	Mean Links	Mean Melds	Peak Heap	Dec-key Rate	Relax Rate	Mean Swaps
dijkstra	binary	30	170680.267	0	0.224	3.197	0	0	339.033	1	0.766	78246.667
dijkstra	fibonacci	30	202682.333	1009.267	0.224	3.197	73371.067	0	339.033	1	0.766	0
dijkstra	pairing	30	138921.433	2504.8	0.224	3.197	136416.433	136416.433	339.033	1	0.766	0
prim	binary	10	276402.8	0	0.421	3.197	0	0	1125.7	1	–	168789.4
prim	fibonacci	10	126259.9	3435.1	0.421	3.197	27501.1	0	1125.7	1	–	0
prim	pairing	10	58445.1	4547.3	0.421	3.197	53732.5	53732.5	1125.7	1	–	0

E C++ Benchmark Summary

This appendix reports compact C++ summaries for the metrics used in the main text. Heap types are shown as columns to avoid repeating each benchmark case three times. The full CSV file contains additional aggregates, but the tables below preserve the runtime and priority-queue-memory values used for the report figures.

E.1 Median Runtime

Table 6: Mean median runtime from `cpp_summary.csv` in milliseconds

Algorithm	Case	Parameters	Binary (ms)	Pairing (ms)	Fibonacci (ms)
Dijkstra	Sparse ER	n=25, m=75	0.007	0.012	0.011
Dijkstra	Sparse ER	n=50, m=150	0.022	0.035	0.034
Dijkstra	Sparse ER	n=100, m=300	0.046	0.078	0.085
Dijkstra	Medium ER	n=25, p=0.2	0.012	0.017	0.015
Dijkstra	Medium ER	n=50, p=0.12	0.023	0.037	0.037
Dijkstra	Medium ER	n=100, p=0.08	0.057	0.091	0.097
Dijkstra	Dense ER	n=25, p=0.35	0.013	0.019	0.018
Dijkstra	Dense ER	n=50, p=0.25	0.031	0.045	0.046
Dijkstra	Dense ER	n=100, p=0.18	0.077	0.115	0.122
Dijkstra	Superdense ER	n=25, p=0.65	0.016	0.022	0.02
Dijkstra	Superdense ER	n=50, p=0.5	0.043	0.058	0.059
Dijkstra	Superdense ER	n=100, p=0.35	0.108	0.148	0.157
Dijkstra	Grid	rows=5, cols=5	0.009	0.013	0.01
Dijkstra	Grid	rows=8, cols=8	0.019	0.032	0.031
Dijkstra	Grid	rows=10, cols=10	0.03	0.053	0.05
Dijkstra	ca-GrQc	largest comp.; 50k-edge cap	2.804	4.996	5.913
Dijkstra	roadNet-PA	largest comp.; 50k-edge cap	5.639	7.79	8.825
Dijkstra	roadNet-CA	largest comp.; 50k-edge cap	5.567	7.992	8.829
Dijkstra	Dijkstra stress	k=1, r=2000	0.795	0.625	0.618
Dijkstra	Dijkstra stress	k=50, r=1000	3.655	2.591	4.492
Dijkstra	Dijkstra stress	k=100, r=2000	9.256	8.416	11.049
Prim	Sparse ER	n=25, m=75	0.012	0.017	0.016
Prim	Sparse ER	n=50, m=150	0.026	0.034	0.034
Prim	Sparse ER	n=100, m=300	0.051	0.069	0.081
Prim	Medium ER	n=25, p=0.2	0.013	0.016	0.015
Prim	Medium ER	n=50, p=0.12	0.026	0.038	0.037
Prim	Medium ER	n=100, p=0.08	0.062	0.083	0.096
Prim	Dense ER	n=25, p=0.35	0.014	0.019	0.017

Table 6: Mean median runtime from `cpp_summary.csv` in milliseconds. Continued.

Algorithm	Case	Parameters	Binary (ms)	Pairing (ms)	Fibonacci (ms)
Prim	Dense ER	n=50, p=0.25	0.032	0.042	0.045
Prim	Dense ER	n=100, p=0.18	0.085	0.107	0.121
Prim	Superdense ER	n=25, p=0.65	0.017	0.023	0.02
Prim	Superdense ER	n=50, p=0.5	0.044	0.053	0.059
Prim	Superdense ER	n=100, p=0.35	0.12	0.136	0.155
Prim	Grid	rows=5, cols=5	0.009	0.013	0.01
Prim	Grid	rows=8, cols=8	0.023	0.029	0.03
Prim	Grid	rows=10, cols=10	0.037	0.046	0.053
Prim	ca-GrQc	largest comp.; 50k-edge cap	3.047	3.862	5.16
Prim	roadNet-PA	largest comp.; 50k-edge cap	6.209	5.867	7.078
Prim	roadNet-CA	largest comp.; 50k-edge cap	6.364	5.984	7.498
Prim	Prim stress	k=1, r=2000	0.794	0.616	0.613
Prim	Prim stress	k=50, r=1000	3.834	2.707	4.743
Prim	Prim stress	k=100, r=2000	10.112	8.924	10.457

E.2 Estimated Priority Queue Memory

Table 7: Mean estimated priority queue memory from `cpp_summary.csv` in KiB

Algorithm	Case	Parameters	Binary (KiB)	Pairing (KiB)	Fibonacci (KiB)
Dijkstra	Sparse ER	n=25, m=75	0.406	1.367	2.062
Dijkstra	Sparse ER	n=50, m=150	0.729	2.734	3.625
Dijkstra	Sparse ER	n=100, m=300	1.391	5.469	6.75
Dijkstra	Medium ER	n=25, p=0.2	0.431	1.367	2.062
Dijkstra	Medium ER	n=50, p=0.12	0.912	2.734	3.625
Dijkstra	Medium ER	n=100, p=0.08	2.391	5.469	6.75
Dijkstra	Dense ER	n=25, p=0.35	0.598	1.367	2.062
Dijkstra	Dense ER	n=50, p=0.25	1.195	2.734	3.625
Dijkstra	Dense ER	n=100, p=0.18	2.391	5.469	6.75
Dijkstra	Superdense ER	n=25, p=0.65	0.598	1.367	2.062
Dijkstra	Superdense ER	n=50, p=0.5	1.195	2.734	3.625
Dijkstra	Superdense ER	n=100, p=0.35	2.391	5.469	6.75
Dijkstra	Grid	rows=5, cols=5	0.273	1.367	2.062
Dijkstra	Grid	rows=8, cols=8	0.5	3.5	4.5
Dijkstra	Grid	rows=10, cols=10	0.741	5.469	6.75
Dijkstra	ca-GrQc	largest comp.; 50k-edge cap	48.242	227.4	260.4
Dijkstra	roadNet-PA	largest comp.; 50k-edge cap	52.238	619.3	708.3
Dijkstra	roadNet-CA	largest comp.; 50k-edge cap	51.684	611.6	699.4
Dijkstra	Dijkstra stress	k=1, r=2000	39.82	109.5	125.6
Dijkstra	Dijkstra stress	k=50, r=1000	36.105	57.477	66.188
Dijkstra	Dijkstra stress	k=100, r=2000	72.207	114.9	131.8
Prim	Sparse ER	n=25, m=75	0.473	1.367	2.062
Prim	Sparse ER	n=50, m=150	0.745	2.734	3.625
Prim	Sparse ER	n=100, m=300	1.391	5.469	6.75
Prim	Medium ER	n=25, p=0.2	0.448	1.367	2.062
Prim	Medium ER	n=50, p=0.12	0.895	2.734	3.625
Prim	Medium ER	n=100, p=0.08	2.391	5.469	6.75
Prim	Dense ER	n=25, p=0.35	0.598	1.367	2.062
Prim	Dense ER	n=50, p=0.25	1.195	2.734	3.625
Prim	Dense ER	n=100, p=0.18	2.391	5.469	6.75
Prim	Superdense ER	n=25, p=0.65	0.598	1.367	2.062
Prim	Superdense ER	n=50, p=0.5	1.195	2.734	3.625
Prim	Superdense ER	n=100, p=0.35	2.391	5.469	6.75
Prim	Grid	rows=5, cols=5	0.285	1.367	2.062
Prim	Grid	rows=8, cols=8	0.625	3.5	4.5
Prim	Grid	rows=10, cols=10	0.941	5.469	6.75

Table 7: Mean estimated priority queue memory from `cpp_summary.csv` in KiB. Continued.

Algorithm	Case	Parameters	Binary (KiB)	Pairing (KiB)	Fibonacci (KiB)
Prim	ca-GrQc	largest comp.; 50k-edge cap	48.242	227.4	260.4
Prim	roadNet-PA	largest comp.; 50k-edge cap	76.238	619.3	708.3
Prim	roadNet-CA	largest comp.; 50k-edge cap	75.684	611.6	699.4
Prim	Prim stress	k=1, r=2000	39.82	109.5	125.6
Prim	Prim stress	k=50, r=1000	36.105	57.477	66.188
Prim	Prim stress	k=100, r=2000	72.207	114.9	131.8